

Pruning the ARL Sensor Interface: a $6.7\times$ Per-Step Speedup with Bit-Identical Trajectories

Josef Leinweber, Jonas Bock • Integrated Energy Systems

1. The cost

The `socal_grid` environment advertises **66,223 sensors and 11,250 actuators**: 9,637 pandapower elements (2,294 buses, 2,595 lines, 2,465 sgens, 1,932 loads, 345 transformers) at roughly seven attributes each. This is a property of the grid model and the `powergrid` module, not of any one run — every experiment in the repository sees the same number. The agents subscribe to **31** of them. The remaining 99.95 % are constructed, serialised, shipped across the message bus and discarded, once per step, for the whole of training.

2. Where the filtering happens



The filter runs *between* enumeration and entity creation, so a pruned sensor is never wired, never stepped and never serialised — the cost is not reduced, it is not incurred. Two properties make this safe rather than merely fast. First, `palaestraAI` validates every agent subscription against the advertised sensors at setup and raises before the first step on any mismatch, so a run that starts has full agent observability *by construction*: the filter cannot silently starve a learner, only abort the run. Second, filtering preserves the relative order of the survivors, so summation order in reward computation — and hence its floating-point result — is unchanged.

3. Validation

Four arms were generated from one source run document by exactly one documented mutation each, then run for four scripted phases $\times$ 60 steps. At every step we compared the SHA-256 of the fire raster, the structural telemetry, and all 1,932 load sensors, and report the number of grid cells whose state differs. Agreement is exact, not within a tolerance.

The `grid_probe` agent was removed from every arm. Its `DummyMuscle` writes `actuator.space.sample()` into `Powergrid-0.0-load-0.0.p_mw` — a random active-power setpoint each step, drawn from a gymnasium `Box` whose RNG is seeded from OS entropy at construction. With it present, two byte-identical configurations diverge, and no comparison between any two arms means anything.

	Arms differ by	Cells differing
Reproducibility control	nothing (identical configurations)	0
Treatment	the filter, on vs. off	0
Sensitivity control	fire wind speed +3.6 %	360

The sensitivity control perturbs the fire driver’s `base_speed` ($14 \rightarrow 14.5$ m/s) with terrain and structure count held identical, and must diverge. It is what gives the two zeros meaning: without it, “identical” reported twice is equally consistent with a comparison that detects nothing at all. Arm equivalence was checked against the *stored* run documents rather than the configuration files — treatment and baseline differ in exactly ten leaves: the run identifier, the derived experiment identifier, and the filter entries in each of the four phases.

4. Result

Running each arm at two lengths (8 and 60 steps) determines $\text{total} = 4(\text{startup} + \text{steps} \times \text{per_step})$ exactly, separating fixed from marginal cost (Fig. 1). Per-step simulation cost falls from 9.77 s to 1.45 s, a factor of **6.73**. Per-phase startup is unchanged ($53.8 \rightarrow 53.4$ s, $1.01\times$). Wall-clock noise between two bit-identical runs was 0.6 %, so the per-step figure is not a scheduling artefact. Because the fixed cost is untouched, any end-to-end ratio depends on episode length and understates the effect; the per-step number is the one that transfers.

5. Limitations

Scripted policies only: the DRL phases still contain `grid_probe`, so they are not reproducible against themselves and bit-comparison does not apply — equivalence for learners rests on the argument in Section 2. The store retains only load sensors (identically in both arms), so equivalence is established on the outcome-relevant projection: the one the grid-outage metric uses. One scenario, one seed.

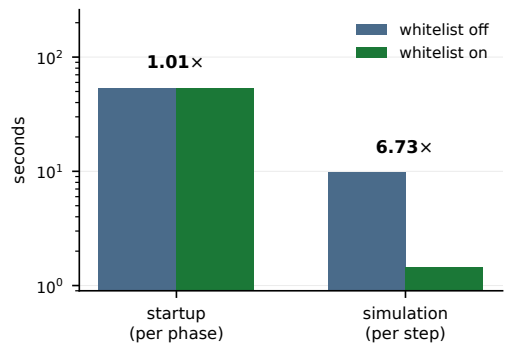


Figure 1: The gain is confined to the per-step term; fixed startup is untouched.